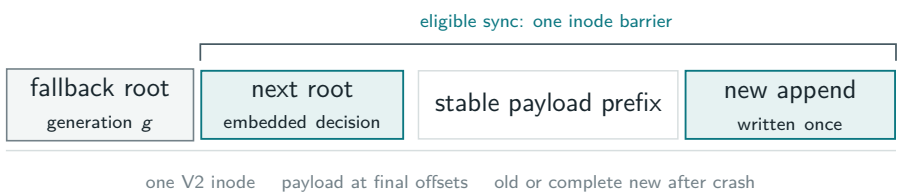
Atomic append-only storage, without writing payload twice

UNO / V2 and the contiguous journal simplification



The contract we actually need

last successfully synchronized epoch

↓

append bytes are visible to this handle

before publication, restart exposes **old**

successful `sync`: complete new epoch

↓

one blob or an all-or-nothing group

after success, restart exposes **new**

FAULT MODEL

Any subset of writes not covered by a completed durability barrier may survive.

INDETERMINATE WINDOW

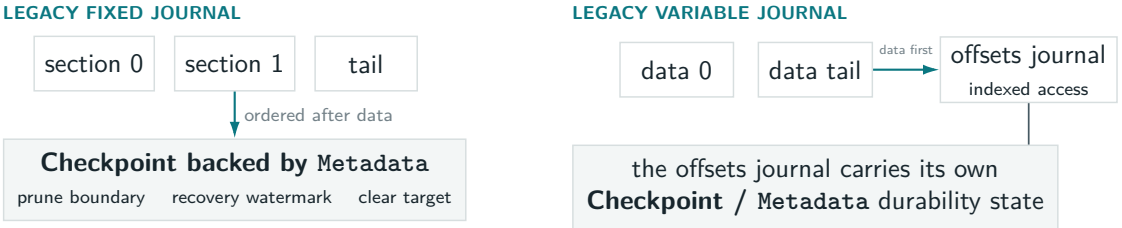
A failed or cancelled sync may recover old or complete new—never a mixture.

PERFORMANCE TARGET

Keep ordinary append behavior: write payload once and pay at the existing sync.

Atomicity is a crash-recovery property; durability remains attached to `sync`.

Before: each journal coordinated its own crash state



- `commit`, `start_sync`, and stronger `sync` kept data, indexes, and recovery hints in a safe order.
- Main had no filesystem primitive for atomically publishing several blob roots together.

The journals were correct, but every compound structure owned a bespoke durability protocol.

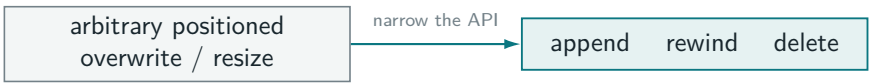
Why generic atomic writes were too expensive

REJECTED DIRECTIONS EXPLORED IN THIS PR—NOT BEHAVIOR SHIPPED ON MAIN

Shadow / redo payload
write new bytes aside, then copy or replay them
≈ 2× payload I/O

Filesystem COW
excellent with reflinks, but not a portable assumption
ext4 cannot be the slow path

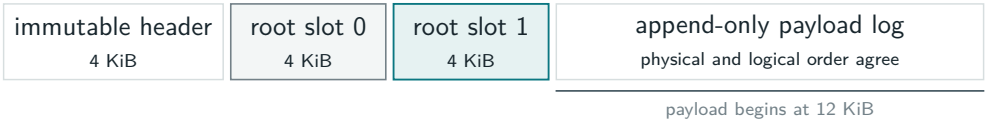
Coordinator file
another manifest, write, barrier, and recovery dependency
extra serialized metadata



A checksum can detect a torn overwrite; it cannot restore overwritten bytes. Append-only preserves the old prefix, so a root can select a complete visible prefix.

The decisive optimization is an API constraint: preserve old bytes and publish a new length.

V2: payload at final offsets, authority in two roots



40-byte root header
spelling generation logical length
12-byte app tag CRC32C

Prepared group witness
root header + magic / length / CRC
+ exact descriptor

APPEND

Write through immediately at the byte's final file offset.

PUBLISH

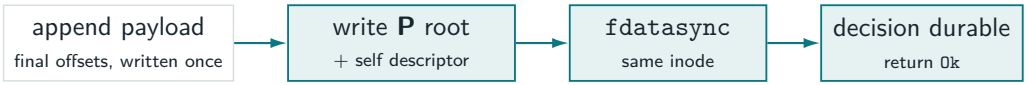
Alternate root slots; generation parity preserves the immediate fallback.

RECOVER

Read bounded roots and, only for an unresolved epoch, checksum a bounded suffix.

No extent map, hole punching, or compaction: a root selects a reachable payload prefix.

One blob: the durable decision fits in one barrier



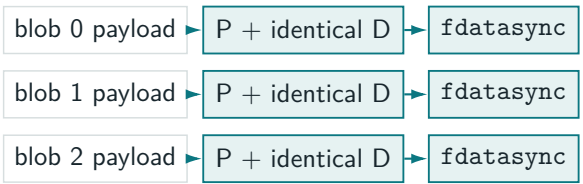
the old root is never overwritten
while publishing its successor

ON RESTART

- If the prepared root, descriptor, and required suffix CRC all validate: expose new.
- Otherwise: expose the preceding committed root.
- No follow-up P→C marker and no second durability barrier.

The descriptor remains the durable decision; later compatible epochs alternate and supersede it.

Several blobs: replicate the decision, not the payload



Canonical descriptor D

exact partition + name
persistent 128-bit incarnation
candidate root and logical length
suffix start + CRC32C
removal operations

Slot bound: at most 28 empty-name participants fit in 4 KiB; real names/removals lower it. Configured cap: 32.

Durable frontier: every barrier completes. Any member leads to exact peers; no coordinator file. Variable data + offsets are one such group.

Current atomic batches are root-slot bounded

EVERY DIRTY PARTICIPANT: ONE 4 KIB ROOT SLOT HOLDS THE COMPLETE DECISION

40 B	16 B	$\leq$ 4,040 B exact group descriptor
candidate root	witness header	paths + incarnations + roots + CRC ranges + removals

HARD ADMISSION LIMIT

32	$\leq$ 28
configured dirty-participant cap	fit even with zero-byte names

Only dirty participants count. Each costs 140 B + partition + blob name; a delete repeats its target. Real names lower 28. Oversize is rejected before roots are staged.

SOFT RECOVERY-WORK BOUND

64 MiB aggregate

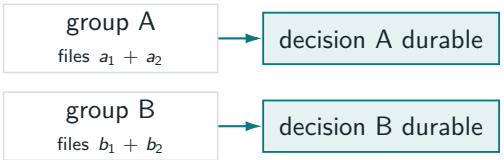
maximum unresolved payload suffix revalidated after crash

Larger or non-contiguous epochs preflush before roots are staged. They are not rejected and payload is not written twice; publication may wait only for the uncovered durability frontier.

Descriptor bytes set the hard ceiling; 64 MiB only bounds recovery work. Contiguous uses at most 2 fixed or 4 variable participants.

Intentional limitation: no global sync order

INDEPENDENT GROUPS



No global order

no root WAL, coordinator, LSN, or commit sequence

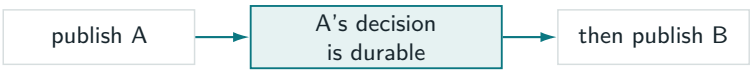
What this avoids

no global lock or volume-like filesystem layer; disjoint groups stay concurrent

CRASH STATES FOR INDEPENDENTLY STARTED A AND B

old A / old B | new A / old B | **old A / new B is possible** | new A / new B

WHEN B HAS A TEMPORAL DEPENDENCY ON A



old/old, new/old, new/new
never old A / new B

Atomic membership is local. Put a temporal dependency in one group, wait at its edge, or add a higher-level WAL only when a total order is required.

Recovery is correct for arbitrary surviving write subsets

What validates after restart	What recovery can prove	Exposed result
No M/T proof; every exact P, descriptor, incarnation, candidate, and required suffix validates	The replicated descriptor names one complete group decision	New roots for the whole group
No M/T proof; any required check is missing or mismatched	The exact group decision is incomplete	Old roots for the whole group
Any exact materialized (M) or tombstone (T) root validates	The group crossed its durable frontier	Roll forward; repair peers and finish removal

TORN ROOTS

Root and witness checksums reject mixed spellings and partial installation. P is invisible to ordinary recovery.

TORN PAYLOAD

A trusted prefix plus CRC32C over the remaining bounded suffix prevents partial visibility.

THREAT BOUNDARY

CRC32C detects crash damage on trusted local disk; it is not tamper authentication.

The protocol never assumes that surviving writes form a prefix in submission order.

Append is the fast path; rewind and delete remain recoverable

APPEND

write final tail once
publish longer root
no relocation or compaction

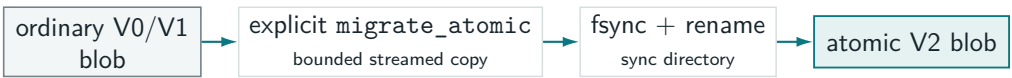
REWIND

publish shorter root
then `ftruncate` in place
restart repeats a lost truncate

DELETE

durable tombstone
then unlink + directory sync
no separate delete manifest

COMPATIBILITY AND MIGRATION

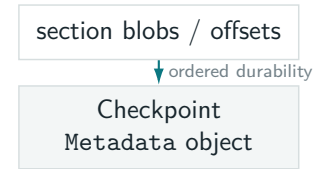


Ordinary opens, V0/V1 formats, and legacy contiguous journals are unchanged. Migration is explicit, idempotent one blob at a time, needs temporary space for one full copy, and does not require reflinks or Linux-only copy commands.

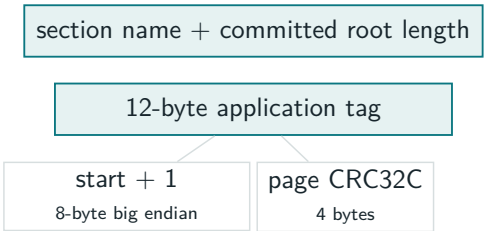
V2 is opt-in; main's ordinary blob behavior is not silently reinterpreted.

Contiguous V2 removes its `Metadata` checkpoint

BEFORE



WITH ATOMIC V2 ROOTS

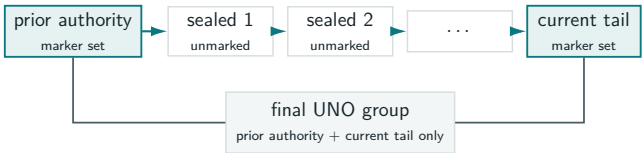


- Variable V2 publishes matching data + offsets roots in the **same UNO group**; neither can advance alone.
- One fixed section or variable pair holds the start; names + root lengths recover the remaining bounds.
- `MutableV2`: consuming mutations, `start_sync`, and `sync`—no legacy `commit` split.

Offsets remain as the variable index; the separate checkpoint object disappears.

Rollover and reopen work stay bounded

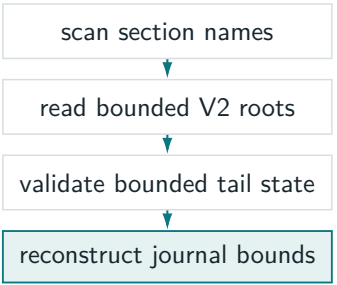
CROSSING MANY FIXED SECTIONS IN ONE EPOCH



Sealed intermediate sections may be durably staged while unreachable; their names fill the interval once the marker publishes.

fixed: **2 roots** variable: **4 roots**
prior/current × data/offsets, all in one group

OPEN / RESTART WORK



Fixed journals may read the final partial checked page of each section. Sealed variable sections are page-complete, so only active data/offset tails need equivalent validation.

Opening scans names and roots—never every frame, historical page, or complete blob.

UNO reaches 94.7% of ordinary-write throughput

PAIRED LINUX / EXT4 HOT-PATH BENCHMARK

5.3% median throughput overhead

Five seeded ratios: `.947`, `.947`, `.923`, `.937`, `.956`
range 92.3–95.6%; median p50 ≈ 0.857 ms vs 0.785 ms ordinary

WORKLOAD

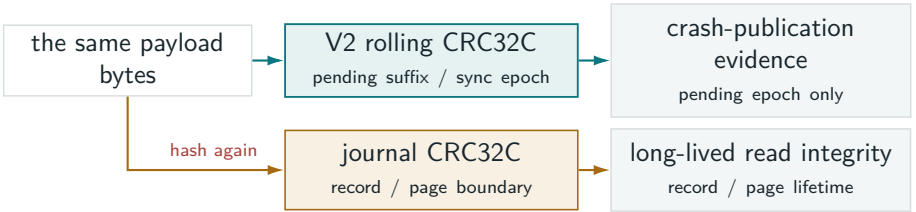
- 4 blobs per group
- one contiguous 64 KiB append per blob
- 256 KiB logical payload per group
- 3,000 groups per side, seeds 71–75
- 2 workers; ordinary and UNO interleaved

What remains non-free
root/descriptor work + group locking; recovery evidence; one barrier per participant

Recovery bound: 64 MiB aggregate suffix; larger or non-contiguous epochs preflush

Payload is written once; the group has no coordinator file; variable data + offsets publish atomically.

Open question: can callers reuse V2's CRC work?



A POSSIBLE BRIDGE

- Accept caller-supplied spans; return composable `(offset, len, CRC32C)` receipts.
- Combine CRCs across write/preflush chunks without rereading bytes.
- Caller maps receipts through its existing index and persists them in the same UNO group.

WHY THIS IS NOT AUTOMATIC

- A write may contain many records; one record may cross writes or preflush ranges.
- Byte offsets alone do not identify application items; variable journals need their offsets index.
- Root tags cannot hold a checksum map; storage-owned boundaries would recreate metadata.

Promising direction: share checksum computation, while callers retain ownership of logical boundaries.